Integration Testing: Testing How Components Work Together

SE 413/513 — Software Testing

Course Reading Material

Unit: Test Levels

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.
For permissions or questions regarding the use of this material, contact hasanka@gvsu.edu.

Please cite [1] when using, adapting, or reproducing this material. The corresponding BibTeX entry is provided below:

```
@misc{hasan2026stesting,
author = {Hasan, Kamrul},
year = {2026},
title = {Software Testing},
institution = {Grand Valley State University},
url = {https://github.com/mdkamrulhasan/software-testing}
}
```

Contents

1 Learning Objectives **4**

2 Where We Are **4**

3 What Is Integration Testing? **5**
3.1 Defects That Live Between Components 5

4 Integration Strategies **6**
4.1 Non-Incremental: Big-Bang Integration 6
4.2 Incremental Integration 7

5 The Running Example **8**

6 Design and API Reviews: Finding Integration Problems Early **9**
6.1 What to Review 9
6.2 Reviewing the Shop 10
6.3 From Review Findings to Test Scenarios 10

7 API Integration Testing **11**
7.1 What to Check 12

7.2 API Examples . . . 12
7.3 Top-Down: The Real API on a Stubbed Service . . . 13
7.4 The Assembled API . . . 14
7.5 Authentication . . . 15
7.6 Common API Integration Failures . . . 15

8 Database Integration Testing **15**
8.1 Which Database Should a Test Use? . . . 16
8.2 Connectivity and Schema . . . 17
8.3 CRUD Operations . . . 17
8.4 Constraints . . . 17
8.5 Transactions and Data Consistency . . . 18
8.6 Back in the Main Suite: Registration . . . 19

9 Service Integration Testing **20**
9.1 The Payment Client . . . 20
9.2 A Fake Provider, Over Real HTTP . . . 21
9.3 Testing Unreliable Dependencies . . . 22
9.4 Strategies for Unavailable or Unreliable Dependencies . . . 23

10 Test Doubles in Integration Testing **23**
10.1 Stubbing at the HTTP Layer with `responses` . . . 24
10.2 Fakes . . . 25
10.3 Contract Testing . . . 25

11 Integration Test Automation **26**
11.1 Organizing Integration Tests . . . 26
11.2 How Many Integration Tests? The Test Pyramid . . . 26
11.3 Running Integration Tests in CI/CD . . . 27

12 Common Integration Failures **28**

13 Putting It Together: A Hands-On Walkthrough **28**

14 Best Practices **29**

15 Common Mistakes **29**

16 Applying This to Your Semester Project **30**

17 A Practical Mental Model **30**

18 Summary **31**

19 Practice Exercises **32**

20 Discussion Questions **32**

21 Key Takeaways for SE 413/513 **33**

22 References **33**

1 Learning Objectives

After completing this reading, you should be able to:

- Explain what integration testing is, why it finds defects that unit testing cannot, and where it sits among the test levels.
- Compare non-incremental (big-bang) and incremental (top-down, bottom-up, sandwich) integration strategies, including the role of stubs and drivers.
- Identify the integration points in a design, and use a design and API review to find likely integration problems *before* running any test.
- Translate review findings into concrete integration test scenarios, and trace each scenario to an automated test.
- Design integration tests for HTTP APIs (request and response validation, status codes, data formats, error handling, authentication), for databases (connectivity, CRUD, constraints, transactions, consistency), and for external services (failures, timeouts, retries).
- Choose where to use real components and where to use stubs, fakes, and mocks in an integration test, and recognize when a test double hides the very defect you are looking for.
- Organize and automate an integration test suite with `pytest`, and run it in a CI/CD pipeline.
- Recognize the common integration failures: incompatible interfaces, incorrect data formats, database inconsistencies, service failures, authentication problems, and unexpected dependency behavior.

2 Where We Are

Week 2 introduced unit testing with `pytest`. Week 4 showed how to isolate a unit from its collaborators with test doubles: dummies, stubs, fakes, spies, and mocks. Weeks 5–8 then covered how to *choose* tests, from the structure of the code (control flow, data flow, coverage) and from the specification (equivalence classes, boundaries, decision tables, state machines, properties).

Almost every test so far has been a *unit* test: it exercises one function or class, and anything that unit talks to is either trivial or replaced by a double. That isolation is what makes unit tests fast and precise. It is also their blind spot.

Key Idea

A unit test checks that a component does what *its author* believes it should do, given what *its author* believes its collaborators do. Integration testing checks whether those beliefs are true: whether the components, once connected, actually agree with each other.

This reading follows one running example throughout: a small online shop's order backend, which talks to a database and to a third-party payment provider. Its code, its interface specifications, a written design review, and 46 tests are in the course repository under `demo-testing-suite/demos/integration_testing/`[1]. Every listing and every output in this reading comes from running those files.

3 What Is Integration Testing?

Integration testing is testing that focuses on the *interactions* between components or systems[2]. Its test basis is not the specification of one function, but the *interfaces* between parts: function signatures and the exceptions they raise, API specifications, database schemas, message formats, and the architecture that connects them[3, 4].

The ISTQB Foundation syllabus distinguishes two kinds[2]:

- **Component integration testing** — the interactions between components *inside* one system: a web handler calling a service, a service calling a repository, a repository issuing SQL.
- **System integration testing** — the interactions between *systems*: your shop and a payment provider, a mail server, a partner's API.

Both appear in this reading.

Level	Question it answers	What is real	Typical speed
Unit	Does this component make the right decisions?	One component; everything else doubled	milliseconds
Integration	Do these components agree with each other?	Two or more components, and often a real database or protocol	milliseconds to seconds
System	Does the whole system meet its requirements?	Everything (Week 10)	seconds to minutes
Acceptance	Is it what the users need?	Everything, judged by users (Week 11)	varies

3.1 Defects That Live Between Components

What kind of defect passes every unit test? One that is not *in* any single component, but in the agreement between two of them. The shop has two deliberately planted examples, and the repository lets you switch either one on with an environment variable:

- `SHOP_PAYMENTS=v1` swaps in a payment client that still sends amounts in the provider's *old* format (a dollars string, `"12.50"`), although the provider now requires integer cents (`1250`).
- `SHOP_SERVICE=buggy` swaps in an order service that commits a stock reservation and the order it belongs to in two separate database transactions.

Here is a good, conventional unit test of the order service (Week 4 style), with every collaborator mocked:

```
@pytest.fixture
def service(repository, payments):
    return OrderService(MagicMock(), repository, payments)  # MagicMock: supports ‘with‘


def test_declined_charge_releases_the_stock(service, repository, payments):
    payments.charge.side_effect = PaymentDeclined("card_declined")
    with pytest.raises(PaymentDeclined):
        service.place_order("MUG", 2, "ana@example.com")
    repository.release_stock.assert_called_once_with("MUG", 2)
    repository.update_order.assert_called_with(1, "payment_failed")
```

Its three tests pass against the correct code — and against both broken versions:

```
$ pytest tests/test_order_service_unit.py
3 passed in 0.10s
$ SHOP_PAYMENTS=v1 pytest tests/test_order_service_unit.py
3 passed in 0.06s
$ SHOP_SERVICE=buggy pytest tests/test_order_service_unit.py
3 passed in 0.06s
```

Nothing is wrong with these tests. A mock has no wire format, so it cannot reject `"12.50"`; it has no transactions, so it cannot leave stock reserved for an order that does not exist. Those defects are only visible when the real parts are connected — which is the job of integration testing. We will find both of them in Sections 8 and 10.

Typical integration defects include[3, 4]:

- **Interface mismatches:** wrong parameter order, units, types, or optional fields.
- **Incorrect data formats:** a number sent as a string, a date in the wrong time zone, an HTML error page where JSON was expected.
- **Wrong assumptions about behavior:** the callee raises where the caller expected `None`; a "create" call is not idempotent; a dependency is slower than assumed.
- **Inconsistent state:** two components that must change together, and do not.
- **Configuration and environment:** credentials, URLs, feature flags, schema versions.

Real-World Case Study

Mars Climate Orbiter (1999). NASA lost the Mars Climate Orbiter as it arrived at Mars on September 23, 1999. The investigation board found the root cause in an *interface* between two pieces of software: a ground software file that reported thruster performance produced its results in English units (pound-force seconds), while the navigation software that consumed them expected metric units (newton-seconds), as the project's Software Interface Specification required[9]. Each component did what its own developers intended. The defect existed only in the connection between them — the same class of defect as this reading's `"12.50"` versus `1250`.

4 Integration Strategies

An integration *strategy* decides in what order components are combined and tested. The classic distinction is between non-incremental and incremental integration[5, 4].

4.1 Non-Incremental: Big-Bang Integration

In **big-bang** integration, all components are built (and perhaps unit tested), then connected all at once and tested as a whole. It needs no extra scaffolding, and for a very small system it can be the quickest option. Its weakness is *fault localization*: when a test fails, the defect could be in any component or any of the connections between them.

The shop shows this directly. With the outdated payment client switched in, the tests of the fully assembled API fail like this:

```
$ SHOP_PAYMENTS=v1 pytest tests/test_api.py --tb=line
tests/test_api.py:20: assert 502 == 201
tests/test_api.py:40: assert 502 == 402
FAILED tests/test_api.py::test_place_an_order_and_fetch_it_back - assert 502 ...
FAILED tests/test_api.py::test_declined_card_is_402_and_the_stock_comes_back
2 failed, 3 passed in 0.17s
```

`502 Bad Gateway` says the payment provider rejected *something*. Was it the API's input validation, the total computed by the service, the client's request format, the authentication header, or the provider itself? The failing test cannot say. The incremental tests in Section 10 can.

4.2 Incremental Integration

In **incremental** integration, components are added one (or a few) at a time, and the growing assembly is tested at each step. When a new failure appears, it is almost certainly in the component or connection just added. Missing components are simulated:

- A **stub** stands in for a component *below* the one being tested: it is called, and returns canned answers.
- A **driver** stands in for a component *above*: it calls the component under test, the way its real caller eventually will. In `pytest`, the test function itself is the driver.

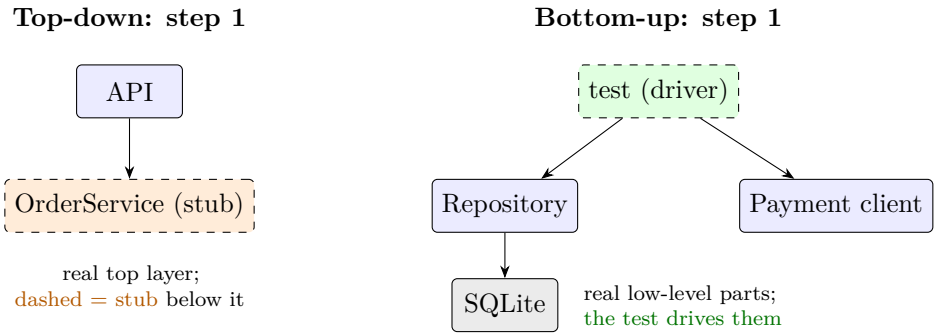


Figure 1: Top-down integration starts from the top layer and stubs what is below it; bottom-up integration starts from the lowest layers and drives them from tests. Each later step replaces a stub (or a driver) with the real component.

Top-down. Start with the top-level component (here, the HTTP API) and stub everything it calls. Replace the stubs with real components layer by layer. Advantages: the overall control flow and the externally visible behavior are tested early, and a working skeleton exists early. Disadvantages: many stubs, and the low-level components — often where the hardest problems live (databases, external services) — are integrated last.

Bottom-up. Start with the lowest-level components (the repository and its database, the payment client and its provider), test them with drivers, then add the components that use them. Advantages: few or no stubs, and the riskiest technical integrations are exercised first. Disadvantages: the system as a whole, as a user would see it, exists only at the end.

Sandwich (hybrid). Work top-down and bottom-up at the same time, and meet in the middle layer. Most real teams do something like this, guided by **risk**: integrate first whatever is newest, least understood, or most expensive to get wrong[5, 2].

Strategy	Scaffolding	Strengths	Weaknesses
Big-bang	none	simple; fine for tiny systems	failures hard to localize; integration starts late
Top-down	stubs	early skeleton; user-visible flow tested early	many stubs; low-level risks found late
Bottom-up	drivers (tests)	risky low-level integrations tested first	whole system visible only at the end
Sandwich / risk-based	both	tests the riskiest connections first	needs planning and judgment

The shop's test files follow a sandwich plan: bottom-up from the database and the payment provider, top-down from the API, meeting at the order service (Section 5).

5 The Running Example

The shop's backend places orders. A request to `POST /orders` reserves stock, charges the customer through a third-party payment provider called PayFast, and records the order. Figure 2 shows its components and their six integration points.

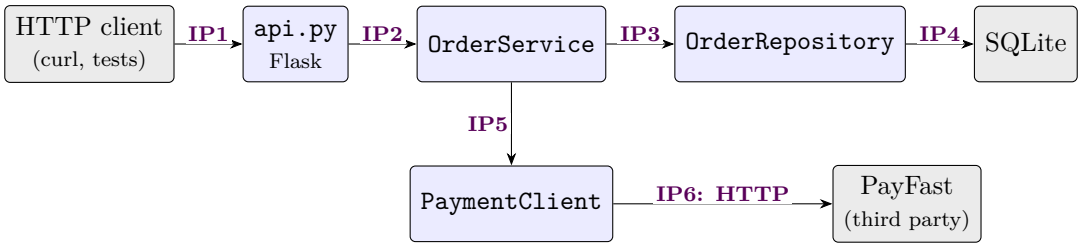


Figure 2: The shop. Grey boxes are outside the shop's own code. Each arrow is an *integration point*: a place where two components must agree.

ID	From	To	Kind	Contract
IP1	HTTP client	`api.py`	HTTP/JSON, API key	`specs/shop_api.md`
IP2	`api.py`	`OrderService`	Python call + exceptions	`service.py`, `errors.py`
IP3	`OrderService`	`OrderRepository`	Python call + transactions	`repository.py`
IP4	`OrderRepository`	SQLite	SQL	schema in `db.py`
IP5	`OrderService`	`PaymentClient`	Python call + exceptions	`payments.py`, `errors.py`
IP6	`PaymentClient`	PayFast	HTTP/JSON, Bearer token	`specs/payment_provider_api.md`

Tip

Listing integration points is the first step of any integration test plan. Follow each arrow in the architecture diagram, and for each one write down *who calls whom*, *over what* (a function call, SQL, HTTP, a queue), and *where the agreement is written down*. An arrow with nothing in the last column is a risk in itself.

The test files map onto the strategies of Section 4:

Test file	What is real	Strategy step
`test_order_service_unit.py`	only the service (all mocked)	unit-test baseline
`test_repository.py`	repository + SQLite (IP4)	bottom-up, step 1 (driver)
`test_payments_stubbed.py`	client; provider stubbed at HTTP	stub at the boundary
`test_payments_live.py`	client + fake provider over HTTP (IP6)	bottom-up, step 1
`test_order_service.py`	service + DB + fake provider (IP3, IP5)	bottom-up, step 2
`test_api_topdown.py`	API; service stubbed (IP1, IP2)	top-down, step 1
`test_api.py`	everything (IP1–IP6)	the assembled system

6 Design and API Reviews: Finding Integration Problems Early

Many integration defects can be found before any integration test runs — in fact before the code is written — by *reviewing* the design and the interface specifications. A **review** is a static test: a systematic examination of a work product by people other than (or in addition to) its author[2, 7]. Fagan's classic inspection work at IBM reported that design and code inspections found a large share of defects before testing began, when they were cheaper to fix[8]. We will study reviews in general later in the course; here we use them for one purpose: *finding integration risks, and turning them into integration tests.*

6.1 What to Review

A design and API review for integration looks at:

- **Component interfaces:** function signatures, parameter names, types, *units*, return values, and the exceptions each component can raise.
- **API specifications:** endpoints, request and response formats, status codes, headers, authentication, versioning.
- **Dependencies:** what each component relies on, how reliable it is, how fast, who owns it, and what happens when it is unavailable.
- **Data flows:** how a value travels across components, and whether it means the same thing at every hop.
- **Architecture:** where transactions begin and end, which state must change together, and which steps cannot be undone.

Area	Review questions
Interfaces	Do caller and callee agree on names, types, units, and which fields are optional?
Data formats	What exactly is on the wire? Numbers, dates, encodings? What does an *error* response look like?
Errors	Is every failure of the callee handled by the caller, and documented?
Authentication	Which credential, in which header? What happens when it is missing, wrong, or expired?
State	What must change together? What if a step fails half-way? Can every step be undone?
Dependencies	How available and how fast is it? Is there a timeout? Is a retry safe?
Data flow	Does each value keep its meaning at every hop?

6.2 Reviewing the Shop

The shop's review was held before its integration tests were written, on the architecture sketch, the two API specifications, and the component signatures. Its record is in `specs/design_review.md`. For example, the reviewer reading PayFast's specification found this row:

Field	Type	Rules
`amount`	integer	**In cents** (minor units). Must be a positive integer: `1250` means $12.50. *Changed in v2: v1 took a decimal string of dollars, `"12.50"`.*

and noted that the shop's payment client was written before v2. That became finding R1. The full set:

#	Finding	IP	Became tests in
R1	PayFast v2 requires integer cents; our client predates v2.	IP6	`test_payments_*`
R2	Nothing checks that the provider receives the same total we store.	IP3,5,6	`test_order_service`
R3	Reserving stock and creating the order must be atomic.	IP3,4	`test_repository`, `test_order_service`
R4	The charge cannot be inside the DB transaction. If it fails, who releases the stock?	IP3,5	`test_order_service`, `test_api`
R5	The provider's 5xx bodies are not guaranteed to be JSON.	IP6	`test_payments_*`
R6	The client retries a `POST` that moves money. Is that safe?	IP6	`test_payments_*`
R7	The first design had no timeout on the provider call.	IP6	`test_payments_live`
R8	Two credentials. A wrong *provider* key is our fault: 502, not "retry later".	IP1,6	`test_api*`, `test_payments_live`
R9	Every service exception needs a documented status; Flask's default errors are HTML.	IP1,2	`test_api_topdown`
R10	SQLite ignores foreign keys unless each connection enables them.	IP4	`test_repository`
R11	Where is input validated? Can bad input reach the service?	IP1,2	`test_api_topdown`

6.3 From Review Findings to Test Scenarios

A review finding is a *hypothesis* about a defect. An integration test is how you confirm or rule it out, and keep it ruled out. Turning one into the other is mechanical once you ask three questions:

1. **Which integration point?** That decides which components the test must make real.
2. **What would go wrong?** That is the scenario: the input or condition that triggers it.

3. **How would we see it?** That is the oracle: an assertion on *both* sides of the interaction where possible.

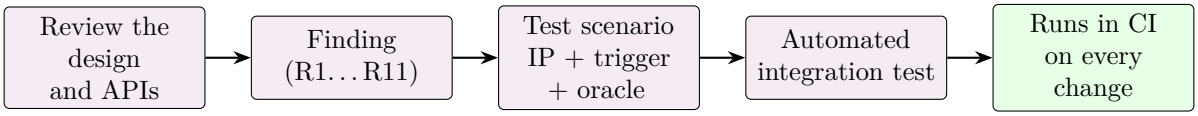


Figure 3: Review findings become test scenarios, then automated tests that keep running.

From Review to Test

R4: "If the charge fails, who releases the stock?"

- **Integration points:** IP3 (service–repository) and IP5 (service–payment client). The database must be real, so committed stock can be observed; the provider can be the fake.
- **Scenario:** the provider declines the card after the stock was reserved and committed.
- **Oracle:** `PaymentDeclined` is raised, the stock read through a *new* database connection is back to 5, and the order's status is `payment_failed`.
- **Test:** `test_order_service.py::test_declined_payment_gives_the_stock_back` (Section 9).

Key Idea

Every review finding should end in one of three places: an integration test that checks it, a design change that removes it, or a documented decision to accept it. A finding that ends nowhere is a defect you have chosen to rediscover later.

The rest of this reading works through the findings by kind of integration: APIs (Section 7), databases (Section 8), and external services (Section 9).

7 API Integration Testing

An **API** (application programming interface) is a contract for how one program may use another. In this section "API" means an HTTP API with JSON bodies, the most common kind between services today. API integration tests check both halves of that contract: that the server honors it for every request a client may send, and that each client sends what the server expects.

7.1 What to Check

Aspect	What an integration test checks
Request validation	Malformed, missing, mistyped, and out-of-range fields are rejected with `400` and a clear message — and never reach the business logic.
Response validation	The body has the documented fields, types, and values; headers such as `Location` and `Content-Type` are present.
Status codes	Each outcome maps to the documented code: `201` created, `400` bad input, `401` not authenticated, `404` not found, `409` conflict, `5xx` server or dependency failure[16].
Data formats	JSON, not HTML, *including for errors*; numbers as numbers; units as specified.
Error handling	Every failure of a lower layer becomes a documented response, not a stack trace.
Authentication	Missing or wrong credentials get `401` (or `403`) and nothing else happens.

The shop's API is specified in `specs/shop_api.md`. Its responses for `POST /orders`:

Status	When	Extra
`201 Created`	The order was placed and paid.	`Location: /orders/<id>`
`400 Bad Request`	Not a JSON object, a missing field, or a rule broken.	
`401 Unauthorized`	`X-API-Key` missing or wrong.	
`402 Payment Required`	The provider declined the card.	stock released
`404 Not Found`	Unknown `sku`.	
`409 Conflict`	Not enough stock.	nothing charged
`502 Bad Gateway`	The provider rejected *our* request (our bug or configuration).	stock released
`503 Service Unavailable`	The provider is down or timed out.	`Retry-After: 30`; stock released

7.2 API Examples

With the shop running locally (`python serve.py`), `curl -i` shows each exchange in full. A successful order, and reading it back:

```
$ curl -i -X POST http://127.0.0.1:8000/orders -H 'X-API-Key: shop-key' \
       -H 'Content-Type: application/json' \
       -d '{"sku": "MUG", "quantity": 2, "customer_email": "ana@example.com"}'
HTTP/1.1 201 CREATED
Content-Type: application/json
Content-Length: 123
Location: /orders/1

{"charge_id":"ch_1","customer_email":"ana@example.com","id":1,"quantity":2,"sku":"
    MUG","status":"paid","total_cents":2500}

$ curl -i http://127.0.0.1:8000/orders/1 -H 'X-API-Key: shop-key'
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 123
```

```
{"charge_id":"ch_1","customer_email":"ana@example.com","id":1,"quantity":2,"sku":"
    MUG","status":"paid","total_cents":2500}
```

Three ways to get it wrong — no API key, a quantity sent as a string, and an out-of-stock product:

```
HTTP/1.1 401 UNAUTHORIZED
Content-Type: application/json

{"error":"missing or invalid API key"}

HTTP/1.1 400 BAD REQUEST
Content-Type: application/json

{"error":"quantity must be a positive integer"}

HTTP/1.1 409 CONFLICT
Content-Type: application/json

{"error":"insufficient stock"}
```

And with the provider down (`python serve.py -provider-mode unavailable`):

```
HTTP/1.1 503 SERVICE UNAVAILABLE
Content-Type: application/json
Retry-After: 30

{"error":"payment provider unavailable"}
```

Transcripts like these are useful testing artifacts in their own right: attached to a defect report, they show exactly what was sent and what came back.

7.3 Top-Down: The Real API on a Stubbed Service

The first API tests are top-down (Section 4): the Flask application is real, and the order service below it is a **stub** that returns a canned order or raises whatever exception the test gives it. No database, no provider:

```python
class StubOrderService:
    """Stands in for OrderService: no database, no provider. Returns a
    canned order, or raises the exception it was given."""

    def __init__(self, raises=None):
        self.raises = raises
        self.calls = []

    def place_order(self, sku, quantity, customer_email):
        self.calls.append((sku, quantity, customer_email))
        if self.raises:
            raise self.raises
        return {"id": 7, "sku": sku, "quantity": quantity, "total_cents": 2500,
                "customer_email": customer_email, "status": "paid", "charge_id": "ch_1"}
```

This makes it cheap to check IP2's half of the contract — every exception the service may raise maps to the documented status (finding R9):

```
@pytest.mark.parametrize("raised, status", [
    pytest.param(UnknownProduct("MUG"), 404, id="unknown-sku"),
    pytest.param(OutOfStock("MUG"), 409, id="out-of-stock"),
    pytest.param(PaymentDeclined("card_declined"), 402, id="declined"),
    pytest.param(PaymentUnavailable("provider returned 503"), 503, id="provider-down"),
    pytest.param(PaymentError("provider rejected the request: 401"), 502, id="our-bug"),
])
def test_service_errors_map_to_the_documented_status(raised, status):
    """R9: every failure the service can raise has a documented status,
    and the body is always JSON."""
    response = api(StubOrderService(raises=raised)).post(
        "/orders", json=GOOD_ORDER, headers=HEADERS)
    assert response.status_code == status
    assert response.is_json and "error" in response.get_json()
```

The stub also *records* its calls, which lets a test check that invalid input never reaches the service at all (finding R11):

```
def test_invalid_bodies_never_reach_the_service(body, message):
    """R11: validate at the boundary."""
    stub = StubOrderService()
    response = api(stub).post("/orders", json=body, headers=HEADERS)
    assert response.status_code == 400
    assert message in response.get_json()["error"]
    assert stub.calls == []
```

Flask's `test_client()` sends requests to the application in-process, through Flask's real routing, request parsing, and response building, without opening a network socket. All 14 top-down tests run in about a tenth of a second.

Common Pitfall

HTML error pages from a JSON API. Web frameworks return HTML for errors they handle themselves — an unknown URL, a wrong HTTP method. A client that calls `response.json()` on that page crashes with a JSON decoding error, far from the real cause. The shop registers JSON handlers for 404 and 405, and `test_unknown_route_answers_in_json_not_html` keeps it that way.

7.4 The Assembled API

Once the layers below are trusted, the same API is tested with every component real except the third-party provider (a fake, over real HTTP; Section 9):

```
def test_place_an_order_and_fetch_it_back(client):
    response = client.post("/orders", json=ORDER, headers=HEADERS)

    assert response.status_code == 201
    assert response.headers["Location"] == "/orders/1"
    assert response.get_json() == {
        "id": 1, "sku": "MUG", "quantity": 2, "total_cents": 2500,
        "customer_email": "ana@example.com", "status": "paid", "charge_id": "ch_1",
    }
```

```
    fetched = client.get(response.headers["Location"], headers=HEADERS)
    assert fetched.status_code == 200
    assert fetched.get_json() == response.get_json()
```

Note the last two lines: the test follows the `Location` header the API itself returned, so it checks that the header is not only present but *usable* — a client–server agreement no unit test sees.

7.5 Authentication

Authentication problems are integration problems by nature: the credential is created in one place, configured in another, and checked in a third. The shop has two (finding R8):

- **Inbound** (IP1): clients send `X-API-Key`. Missing or wrong: `401`, checked before anything else runs.
- **Outbound** (IP6): the shop sends `Authorization: Bearer <secret>` to PayFast. If PayFast answers `401`, the *shop* is misconfigured. Telling the customer "try again in 30 seconds" (`503`) would be wrong; it will fail every time. So it is a `502`, and it is not retried:

```
def test_wrong_provider_key_is_502_not_503(client, payments, monkeypatch, stock):
    """R8: deployed with the wrong PayFast key. That is our fault, and
    "try again in 30 seconds" would be the wrong advice."""
    monkeypatch.setattr(payments, "api_key", "sk_wrong_key")
    response = client.post("/orders", json=ORDER, headers=HEADERS)
    assert response.status_code == 502
    assert stock("MUG") == 5
```

7.6 Common API Integration Failures

Failure	Example	Test that catches it
Field renamed or removed	server sends `total`, client reads `total_cents`	assert the full response body
Type or unit mismatch	`"2"` vs. `2`; `"12.50"` vs. `1250`	invalid-type requests; contract check
Undocumented status	a new exception surfaces as `500`	parametrize over every exception
Error body in another format	HTML 404 page from a JSON API	assert `is_json` on errors
Missing or wrong header	no `Location`; wrong `Content-Type`	assert headers; follow `Location`
Auth misconfiguration	wrong key, wrong scheme (`Token` vs. `Bearer`)	tests with missing and wrong credentials
Version drift	client built against v1, server on v2	contract tests (Section 10)

8 Database Integration Testing

Nearly every application stores data, and the database is the most common dependency an integration test needs to make real. A mock of a repository returns whatever the test tells it to. A real database also enforces types and constraints, assigns ids, isolates transactions, and rolls back on failure — and the application's correctness depends on all of that.

8.1 Which Database Should a Test Use?

Option	Good for	Watch out for
Mock of the repository	unit tests of the caller's decisions	no SQL, no constraints, no transactions
In-memory database (`sqlite3 ":memory:"`)	fast tests of SQL and constraints	only one connection can see it
Temporary database file per test (`tmp_path`)	SQL, constraints, *and* multiple connections	a little slower; needs cleanup (pytest does it)
Same engine as production, in a container	dialect-specific behavior (PostgreSQL, MySQL)	slower; needs Docker in CI

The shop is built on SQLite, so its tests use the production engine: a fresh database *file* per test, under pytest's `tmp_path`. A file (rather than `":memory:"`) lets a test open a *second* connection to check what was really committed:

```
@pytest.fixture
def connection(db_path):
    connection = db.connect(db_path)
    db.create_schema(connection)
    repository = OrderRepository(connection)
    with connection:
        repository.add_product("MUG", "Coffee mug", price_cents=1250, stock=5)
        repository.add_product("TEE", "T-shirt", price_cents=2000, stock=0)
    yield connection
    connection.close()


@pytest.fixture
def stock(db_path):
    """Read a product's stock through a *new* connection: this sees only
    what was committed, not what is pending inside a transaction."""
    def read(sku):
        fresh = db.connect(db_path)
        try:
            return fresh.execute(
                "SELECT stock FROM products WHERE sku = ?", (sku,)
            ).fetchone()["stock"]
        finally:
            fresh.close()
    return read
```

Key Idea

Isolation. Every test gets its own database, created from the application's own schema and seeded with exactly the rows it needs. No test depends on another test's leftovers, so tests can run in any order, or in parallel. Shared, long-lived test databases are the most common cause of integration tests that pass alone and fail together.

8.2 Connectivity and Schema

The first database tests check that the application can connect and that its schema is what the code expects:

```
def test_schema_has_both_tables(connection):
    rows = connection.execute("SELECT name FROM sqlite_master WHERE type = 'table'")
    assert {row["name"] for row in rows} == {"products", "orders"}


def test_create_schema_is_safe_to_run_again(connection):
    db.create_schema(connection)  # e.g. on every application start
    assert connection.execute("SELECT COUNT(*) FROM products").fetchone()[0] == 2
```

8.3 CRUD Operations

The repository is the bottom of the shop, so its tests are bottom-up: the test functions are the *drivers*, calling the repository the way the service eventually will. One test walks an order through create, read, update, and delete:

```
def test_order_create_read_update_delete(connection, repository):
    with connection:
        order_id = repository.create_order("MUG", 2, 2500, "ana@example.com")
    assert repository.get_order(order_id) == {
        "id": order_id, "sku": "MUG", "quantity": 2, "total_cents": 2500,
        "customer_email": "ana@example.com", "status": "pending", "charge_id": None,
    }

    with connection:
        repository.update_order(order_id, "paid", "ch_9")
    assert repository.get_order(order_id)["status"] == "paid"
    assert repository.get_order(order_id)["charge_id"] == "ch_9"

    with connection:
        assert repository.delete_order(order_id)
    assert repository.get_order(order_id) is None
```

Notice what this checks that a mock could not: the column defaults (`status` starts as `"pending"`, `charge_id` as `None`), the generated id, and that each SQL statement is correct for the real schema.

8.4 Constraints

A schema is part of the application's logic. The shop's schema says stock can never be negative, and an order must refer to a real product:

```
CREATE TABLE IF NOT EXISTS products (
    sku            TEXT PRIMARY KEY,
    name           TEXT NOT NULL,
    price_cents    INTEGER NOT NULL CHECK (price_cents > 0),
    stock          INTEGER NOT NULL CHECK (stock >= 0)
);
CREATE TABLE IF NOT EXISTS orders (
    id             INTEGER PRIMARY KEY,
```

```
    sku             TEXT NOT NULL REFERENCES products (sku),
    ...
```

Finding R10 came from reading SQLite's documentation during the review: SQLite does *not* enforce `REFERENCES` unless each connection turns foreign keys on[21]. `db.connect()` does so, and a test makes sure it keeps doing so:

```
def test_database_rejects_an_order_for_an_unknown_product(connection, repository):
    """R10: SQLite ignores foreign keys unless each connection turns them
    on. db.connect() does; this test fails if that line is removed."""
    with pytest.raises(sqlite3.IntegrityError, match="FOREIGN KEY constraint failed"):
        with connection:
            repository.create_order("NOPE", 1, 100, "ana@example.com")
```

Deleting the `PRAGMA foreign_keys = ON` line from `db.py` makes exactly this test fail. That is the point of turning a review finding into a test: the next person who "cleans up" that line finds out immediately.

8.5 Transactions and Data Consistency

A **transaction** groups several statements so that they take effect together or not at all. In Python's `sqlite3`, a `with connection:` block commits on success and rolls back if an exception escapes it. Two tests pin down that behavior with the real database:

```
def test_uncommitted_changes_are_invisible_to_other_connections(connection, repository,
    stock):
    repository.reserve_stock("MUG", 2)   # opens a transaction; not committed yet
    assert stock("MUG") == 5             # another connection still sees 5
    connection.commit()
    assert stock("MUG") == 3


def test_rollback_undoes_every_statement_in_the_transaction(connection, repository, stock
    ):
    """R3: stock and order are saved together or not at all."""
    with pytest.raises(RuntimeError):
        with connection:
            repository.reserve_stock("MUG", 2)
            repository.create_order("MUG", 2, 2500, "ana@example.com")
            raise RuntimeError("something failed before the commit")
    assert stock("MUG") == 5
    assert count_orders(connection) == 0
```

The order service relies on exactly this. Reserving stock and creating the order happen in *one* transaction:

```
        # One transaction: the stock and the pending order are saved
        # together, or (if anything raises) not at all.
        with self.connection:
            if not self.repository.reserve_stock(sku, quantity):
                raise OutOfStock(sku)
            order_id = self.repository.create_order(
                sku, quantity, total_cents, customer_email
```

```
            )
```

The buggy version (`SHOP_SERVICE=buggy`) splits it into two:

```
        with self.connection:  # KNOWN BUG: commits the stock on its own...
            if not self.repository.reserve_stock(sku, quantity):
                raise OutOfStock(sku)
        with self.connection:  # ...before the order insert can fail
            order_id = self.repository.create_order(
                sku, quantity, total_cents, customer_email
            )
```

It looks harmless, and every mock-based unit test passes (Section 3.1). The integration test makes the database reject the order — `customer_email` is `NOT NULL` — and then checks the stock through a second connection:

```
def test_order_the_database_rejects_leaves_stock_untouched(service, provider, stock):
    """R3: the database refuses the order (NOT NULL customer_email), so
    the stock reservation in the same transaction must be undone too."""
    with pytest.raises(sqlite3.IntegrityError, match="NOT NULL"):
        service.place_order("MUG", 2, customer_email=None)
    assert stock("MUG") == 5
    assert provider.requests == []
```

```
$ SHOP_SERVICE=buggy pytest tests/test_order_service.py --tb=short
tests/test_order_service.py:61: in
    test_order_the_database_rejects_leaves_stock_untouched
    assert stock("MUG") == 5
E   AssertionError: assert 3 == 5
FAILED tests/test_order_service.py::
    test_order_the_database_rejects_leaves_stock_untouched
1 failed, 4 passed in 0.19s
```

Two mugs have left the stock, and no order exists to account for them. In production this is the kind of inconsistency that surfaces weeks later as "inventory doesn't match orders".

Common Pitfall

The end-to-end tests do not catch it either. Run `SHOP_SERVICE=buggy pytest tests/test_api.py`: all five pass, because the API rejects a missing email with `400` before the service is called. Higher-level tests are not a superset of lower-level ones. The service is used by other callers too (scripts, future endpoints), and only a test at the service's own integration point exercises its transaction boundaries.

8.6 Back in the Main Suite: Registration

The course's main `demo-testing-suite` has the same shape of problem. In Week 4, `UserRegistrationService` was tested with a *mocked* `UserRepository`. This week adds `tests/integration/`, which assembles it from the real repository, a real SQLite file, and the application's own schema — which, it turned out, had never existed in the application: until now the `users` table was defined only in a test fixture. Writing the integration tests forced a real `create_schema()` into `app/accounts/user_repository.py`.

One of the new tests documents a defect the mocks could never show:

```
@pytest.mark.xfail(
    strict=True,
    reason="KNOWN ISSUE (Week 9): register() commits the user before sending "
           "the welcome email, with no compensation if the email fails. ..."
)
def test_failed_welcome_email_does_not_leave_a_half_registered_user(connection,
    saved_names):
    service = UserRegistrationService(UserRepository(connection), FailingEmailClient())
    with pytest.raises(ConnectionError):
        service.register("Carol", "carol@example.com")

    assert saved_names() == []
```

If the mail server is down, Carol is saved but never welcomed — and cannot try again, because a second `register("Carol", ...)` is rejected as a duplicate. As with the late-fee bug of Week 1 and the shipping bug of Week 3, it is documented with a strict `xfail` rather than silently fixed (Exercise 4 asks you to fix it).

9 Service Integration Testing

An **external service** is a dependency your team does not own and cannot change: a payment provider, a mail service, a maps API, another team's microservice. Testing against one raises problems a database does not:

- It may be **unavailable** (maintenance, outages), **slow**, or **rate-limited**.
- Calling it may have **real-world effects** (charging a card, sending an email) or **cost money**.
- Its behavior may **change** without your code changing (a new API version).
- You **cannot easily make it fail** on demand, yet its failures are exactly what you most need to test.

9.1 The Payment Client

```
    def charge(self, amount_cents, reference):
        """Charge `amount_cents` (an int, in cents); return the charge id.

        Retries on a 5xx or a network failure. The Idempotency-Key header
        makes that safe: PayFast charges a given key at most once.
        """
        payload = {"amount": amount_cents, "currency": "USD", "reference": reference}
        headers = {
            "Authorization": f"Bearer {self.api_key}",
            "Idempotency-Key": reference,
        }
        for _ in range(self.retries + 1):
            try:
                response = self.session.post(
                    f"{self.base_url}/v1/charges",
                    json=payload, headers=headers, timeout=self.timeout,
```

```
                )
            except (requests.ConnectionError, requests.Timeout) as exc:
                problem = f"provider unreachable ({type(exc).__name__})"
                continue
            if response.status_code in RETRYABLE:
                problem = f"provider returned {response.status_code}"
                continue
            break
        else:
            raise PaymentUnavailable(problem)

        if response.status_code == 201:
            return response.json()["id"]
        if response.status_code == 402:
            raise PaymentDeclined(response.json()["error"])
        raise PaymentError(
            f"provider rejected the request: {response.status_code} "
            f"{response.text.strip()}"
        )
```

Almost every line answers a review finding: the `timeout` (R7), the retry on 5xx and network errors, the `Idempotency-Key` that makes the retry safe (R6), never parsing a 5xx body (R5), and treating a 400 or 401 as *our* error, not retried (R8).

9.2 A Fake Provider, Over Real HTTP

The shop's tests replace PayFast with a **fake**: a small but genuinely working implementation of PayFast's published contract, written with Flask in about 60 lines (`fakes/payment_provider.py`). Unlike a mock, it *enforces* the contract: a wrong key gets a `401`, an amount that is not integer cents gets a `400`, and a repeated `Idempotency-Key` is not charged twice. It can also misbehave on demand: `provider.config["MODE"]` set to `"decline"` or `"unavailable"` (a `503` with an HTML body, as a proxy might send), and `provider.config["DELAY"]` for slow responses.

A fixture runs it on a free local port, so the real `PaymentClient` talks to it over real HTTP, through the real `requests` library:

```
@pytest.fixture
def provider():
    """The fake PayFast, served over real HTTP on a free port."""
    app = create_provider(secret=PROVIDER_SECRET)
    server = make_server("127.0.0.1", 0, app, threaded=True)
    thread = threading.Thread(
        target=server.serve_forever, kwargs={"poll_interval": 0.01}, daemon=True
    )
    thread.start()
    app.url = f"http://127.0.0.1:{server.port}"
    yield app
    server.shutdown()
    thread.join()
```

Port `0` asks the operating system for any free port, so parallel test runs never collide.

9.3 Testing Unreliable Dependencies

With a controllable provider, the failures that are hardest to produce against the real service become ordinary tests:

```
def test_wrong_api_key_is_our_error_and_is_not_retried(provider):
    """R8: a 401 means the shop is misconfigured. Retrying cannot help."""
    misconfigured = PaymentClient(provider.url, "sk_wrong_key", timeout=1.0)
    with pytest.raises(PaymentError, match="401") as caught:
        misconfigured.charge(1250, reference="order-1")
    assert type(caught.value) is PaymentError   # not PaymentUnavailable
    assert len(provider.requests) == 1


def test_outage_is_retried_then_reported_as_unavailable(payments, provider):
    """R5: the 503 body is an HTML page; the client must not try to
    parse it as JSON."""
    provider.config["MODE"] = "unavailable"
    with pytest.raises(PaymentUnavailable, match="503"):
        payments.charge(1250, reference="order-1")
    assert len(provider.requests) == 2


def test_slow_provider_times_out(provider):
    """R7: without a timeout, this call would wait as long as PayFast does."""
    provider.config["DELAY"] = 0.5
    impatient = PaymentClient(provider.url, "sk_test_123", timeout=0.2)
    with pytest.raises(PaymentUnavailable, match="ReadTimeout"):
        impatient.charge(1250, reference="order-1")


def test_same_reference_is_charged_only_once(payments, provider):
    """R6: what makes the client's retries safe."""
    first = payments.charge(1250, reference="order-1")
    second = payments.charge(1250, reference="order-1")
    assert first == second
    assert len(provider.charges) == 1
```

The next level up checks what the *service* does when the provider fails (finding R4). The charge cannot be part of the database transaction — it happens in someone else's system — so the service *compensates*: it undoes its own side by hand.

```
        try:
            charge_id = self.payments.charge(total_cents, reference=f"order-{order_id}")
        except PaymentError:
            with self.connection:
                self.repository.release_stock(sku, quantity)
                self.repository.update_order(order_id, "payment_failed")
            raise
```

```
def test_declined_payment_gives_the_stock_back(service, provider, stock):
    """R4: the charge failed after the stock was committed; the service
    must undo its own side by hand."""
    provider.config["MODE"] = "decline"
```

```
    with pytest.raises(PaymentDeclined):
        service.place_order("MUG", 2, "ana@example.com")
    assert stock("MUG") == 5
    assert service.get_order(1)["status"] == "payment_failed"
```

And data flow across all three components (finding R2): the amount PayFast received is the total the shop recorded.

```
def test_successful_order_charges_exactly_the_order_total(service, provider, stock):
    """R2: the amount that flows to PayFast is the amount we recorded."""
    order = service.place_order("MUG", 2, "ana@example.com")

    assert order["status"] == "paid"
    assert order["charge_id"] == "ch_1"
    assert order["total_cents"] == 2500
    assert provider.charges[0]["amount"] == order["total_cents"]
    assert stock("MUG") == 3
```

Common Pitfall

A timeout is not a failure; it is an unknown. When `test_slow_provider_times_out` gives up after 0.2 seconds, the fake provider is still processing the request, and will charge the card a moment later. The real world is the same. The shop currently treats a timeout like an outage (release the stock, mark the order `payment_failed`), which can leave a paid charge with no order. Production systems *reconcile*: they later ask the provider what happened to that `Idempotency-Key`. The review recorded this as an open item, a documented decision rather than a forgotten one.

9.4 Strategies for Unavailable or Unreliable Dependencies

Strategy	How	Trade-off
Stub at the HTTP layer	`responses` (Section 10)	fast; only as right as its author
Fake service	a small working implementation (`fakes/`)	enforces the contract; must be maintained
Provider sandbox	the vendor's test environment	real behavior; slow, shared, sometimes down
Contract tests	check both sides against a shared contract	catches drift; needs both teams
Timeouts, retries, idempotency	in the client code itself	the *product* copes, not just the tests
Circuit breaker	stop calling a failing service for a while[17]	protects the system; more states to test

10 Test Doubles in Integration Testing

Week 4 introduced test doubles for *unit* testing, where everything except the unit is replaced[10]. In integration testing the question is narrower: *which* components to replace, and with what. The rule of thumb: make real every component on the integration point you are testing, and double

everything beyond it — especially things that are slow, costly, non-deterministic, or have real-world effects.

Test file	API	Service	Database	PayFast
`test_order_service_unit`	—	real	mock	mock
`test_repository`	—	—	real	—
`test_payments_stubbed`	—	—	—	stub (`responses`)
`test_payments_live`	—	—	—	fake, real HTTP
`test_order_service`	—	real	real	fake, real HTTP
`test_api_topdown`	real	stub	—	—
`test_api`	real	real	real	fake, real HTTP

10.1 Stubbing at the HTTP Layer with `responses`

The `responses` library[20] intercepts every call made through `requests`. The client's real code runs — building the request, handling status codes, retrying — but nothing leaves the machine. Here is the test many teams write first:

```
@responses.activate
def test_loose_stub_only_checks_the_return_value(client):
    """The usual first attempt. It cannot fail on a wrong request."""
    responses.post(CHARGES, json={"id": "ch_1", "status": "succeeded"}, status=201)
    assert client.charge(1250, reference="order-1") == "ch_1"
```

And here is a test that checks the *request* against the provider's published contract — finding R1, made executable:

```
@responses.activate
def test_request_matches_the_provider_contract(client):
    """R1, R6, R8: the request itself, checked against
    specs/payment_provider_api.md -- integer cents, Bearer auth, and an
    Idempotency-Key."""
    responses.post(CHARGES, json={"id": "ch_1", "status": "succeeded"}, status=201)
    client.charge(1250, reference="order-1")

    sent = responses.calls[0].request
    assert json.loads(sent.body) == {
        "amount": 1250, "currency": "USD", "reference": "order-1",
    }
    assert sent.headers["Authorization"] == "Bearer sk_test_123"
    assert sent.headers["Idempotency-Key"] == "order-1"
```

Now switch in the outdated client that still sends dollars:

```
$ SHOP_PAYMENTS=v1 pytest tests/test_payments_stubbed.py --tb=short
tests/test_payments_stubbed.py:48: in test_request_matches_the_provider_contract
    assert json.loads(sent.body) == {
E   AssertionError: assert {'amount': '1...e': 'order-1'} == {'amount': 12...e': '
    order-1'}
E     Omitting 2 identical items, use -vv to show
E     Differing items:
E     {'amount': '12.50'} != {'amount': 1250}
FAILED tests/test_payments_stubbed.py::test_request_matches_the_provider_contract
1 failed, 4 passed in 0.14s
```

The loose stub passed — it is one of the four. The contract check names the exact field and both values. Against the fake provider, the same defect is reported in the provider's own words:

```
$ SHOP_PAYMENTS=v1 pytest tests/test_payments_live.py --tb=line
E   shop.errors.PaymentError: provider rejected the request: 400 {"error":"amount
    must be a positive integer number of cents"}
FAILED tests/test_payments_live.py::test_charge_succeeds_in_integer_cents - s...
FAILED tests/test_payments_live.py::test_declined_card - shop.errors.PaymentE...
FAILED tests/test_payments_live.py::test_same_reference_is_charged_only_once
3 failed, 3 passed in 0.59s
```

Compare these with the big-bang failure of Section 4, `assert 502 == 201`: the same defect, but the incremental tests point straight at IP6.

Common Pitfall

Mocks and stubs that agree with the bug. A test double encodes its author's beliefs about the other side. If the same person, with the same outdated belief, writes both the client and its stub, the test confirms the belief instead of checking it. Write doubles from the *other side's* specification, not from your own code; check requests as well as responses; and keep at least one test against something that enforces the contract (a fake, a sandbox, or a contract test).

10.2 Fakes

A **fake** is a working implementation that takes a shortcut unsuitable for production[10]: an in-memory store instead of a database, or the 60-line PayFast above. The main suite uses a fake for email, because sending real email from a test suite is never acceptable:

```
class RecordingEmailClient:
    """A fake email client: works, but only records messages."""

    def __init__(self):
        self.sent = []

    def send(self, to, message):
        self.sent.append((to, message))
```

Fakes are more work than stubs, but they are *shared*: one fake serves every test that touches that dependency, and it can be checked against the real service occasionally to make sure it has not drifted.

10.3 Contract Testing

A **contract test** checks that a test double still behaves like the real thing[11]. In *consumer-driven* contract testing (with tools such as Pact[13]), the consumer's tests record the requests it sends and the responses it expects; the provider's own test suite then replays those requests against the real provider and verifies the responses. Each team runs its own tests, yet a change on either side that breaks the agreement fails a build. `test_request_matches_the_provider_contract` is the consumer half of such a contract, written by hand.

11 Integration Test Automation

11.1 Organizing Integration Tests

- **Separate them on disk.** The main suite now has `tests/integration/` next to the unit-test folders. Anyone browsing the suite can see which tests assemble real components.
- **Mark them.** A registered marker lets anyone run either kind on its own:

```
# pytest.ini
markers =
    unit: fast, isolated unit tests with no I/O
    slow: tests slow enough to skip during rapid local iteration
    integration: tests that assemble real components (database, services) -- Week 9
```

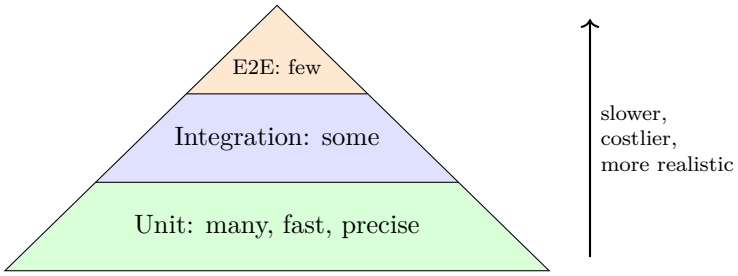
```
# at the top of tests/integration/test_registration.py
pytestmark = pytest.mark.integration
```

```
$ pytest -m integration
3 passed, 75 deselected, 1 xfailed in 0.11s
$ pytest -m "not integration"
1 failed, 72 passed, 4 deselected, 2 xfailed in 0.14s
```

(The one failure and two `xfail`s are the intentional Week 1 and Week 3 bugs, unchanged.)

- **Build the system in fixtures.** Each `conftest.py` assembles real parts (database, fake provider, service, API client) once, so every test reads like a short scenario. Fixtures that depend on fixtures (`client` ← `service` ← `payments` ← `provider`) mirror the architecture.
- **Isolate every test.** A fresh database file, a fresh provider on a fresh port, no shared global state.
- **Name tests after behavior and trace them to findings.** Each shop test's docstring names the review finding it checks (`R1`–`R11`); the review document names the tests.

11.2 How Many Integration Tests? The Test Pyramid



The **test pyramid**[14, 15] recommends many fast unit tests, fewer integration tests, and very few end-to-end tests. The shop follows it: 14 top-down API tests with a stub cost nothing, while only five tests exercise the whole assembly. Its whole suite of 46 tests runs in about a second, because the slowest dependency (the payment provider) is a local fake and each database is a small local file.

11.3 Running Integration Tests in CI/CD

In a continuous integration pipeline, every push runs the tests automatically. A common layout runs the fast unit tests first, and the integration tests only if those pass:

```
# .github/workflows/tests.yml (sketch)
jobs:
  unit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.11"}
      - run: pip install -r requirements.txt
      - run: pytest -m "not integration"
  integration:
    needs: unit                  # only if the unit tests passed
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.11"}
      - run: pip install -r requirements.txt
      - run: pytest -m integration
```

Projects that integrate with a database server rather than SQLite typically start one as a *service container* for the integration job. We return to pipelines in the Continuous Testing week.

Tip

Flaky integration tests — tests that sometimes pass and sometimes fail on the same code — usually come from something shared or something timed: a fixed port another process is using, a database left over from a previous test, a real network call, a sleep that is "usually long enough". The shop avoids each one: port `0`, a database per test, a local fake instead of the network, and timeouts measured against a deliberately slow fake rather than a guessed delay.

12 Common Integration Failures

Failure	Example in this reading	Found by	Could a review find it?
Incompatible interfaces	service exceptions vs. API status codes	top-down API tests with a stub	yes: compare the exception list with the status table
Incorrect data formats	`"12.50"` dollars vs. `1250` cents	request contract check; fake provider	yes: R1, from the provider's changelog
Database inconsistencies	stock reserved, order never created	service + real DB, read via a second connection	yes: R3, "what must change together?"
Service failures	provider down; HTML 503 body	fake provider in `"unavailable"` mode	yes: R5, "what does an error look like?"
Authentication problems	wrong provider key returned as "retry later"	wrong-key tests at client and API	yes: R8, two credentials
Unexpected dependency behavior	slow responses; a timeout that still charges; SQLite ignoring foreign keys	slow fake; FK test	partly: R7, R10; the timeout outcome stays an open item
Partial failure across systems	user saved, welcome email failed	main-suite integration test (`xfail`)	yes: "what if a step fails half-way?"

Every row except the last was found twice in this reading: first on paper, in the design review, and then by an automated test. That is the pattern to aim for.

13 Putting It Together: A Hands-On Walkthrough

To plan integration tests for a new feature, work through five steps. As an example, suppose the shop adds **refunds**: `POST /orders/<id>/refund` calls PayFast's `POST /v1/refunds` and returns the stock.

Step 1 — Identify the integration points. Extend the architecture diagram: the API gains a route (IP1), the service gains `refund_order` (IP2), which updates the order and the stock (IP3, IP4) and calls a new client method (IP5) that calls a new PayFast endpoint (IP6).

Step 2 — Review the design and the APIs. Walk the checklist of Section 6 over each point. Typical findings: *Is a partial refund allowed, and in what units? What if the order is not* `paid`*? What if PayFast is down — is the stock returned anyway? Is a retried refund safe?*

Step 3 — Design test cases. Write each finding as a scenario with an integration point, a trigger, and an oracle on both sides. An integration test case specification is a useful artifact for review and for the semester project:

ID	IP	Precondition	Action	Expected (both sides)
IT-R1	1–6	order 1 `paid`, stock 3	refund order 1	`200`; status `refunded`; stock 5; PayFast has one refund of 2500
IT-R2	2,3	order 1 `payment_failed`	refund order 1	`409`; nothing sent to PayFast; stock unchanged
IT-R3	5,6	PayFast `unavailable`	refund order 1	`503`; status still `paid`; stock unchanged
IT-R4	6	PayFast fails once, then succeeds	refund order 1	one refund at PayFast, same `Idempotency-Key` on both requests
IT-R5	1	no API key	refund order 1	`401`; service never called

Step 4 — Choose real components and doubles. Real API, service, repository, and database; PayFast as the fake (extended with `/v1/refunds`); a `responses` contract check on the refund request's format.

Step 5 — Automate. Add the tests next to the existing ones, reuse the fixtures, name each test after its behavior and its finding, and let CI run them on every push. Exercise 3 asks you to do this.

14 Best Practices

1. **Start from the integration points**, not from the components. List every arrow; test each one deliberately.
2. **Review interfaces before testing them.** Specifications, signatures, units, error formats, and transaction boundaries are cheapest to fix on paper.
3. **Turn every review finding into a test, a change, or a documented decision.**
4. **Integrate incrementally, by risk.** Connect the newest and riskiest parts first, so a failure points at what just changed.
5. **Make real what you are testing; double what is beyond it.** Prefer fakes that enforce the other side's contract over stubs that only return values.
6. **Assert on both sides of an interaction.** The response *and* the database; the order *and* the charge.
7. **Test failures, not just success:** outages, timeouts, wrong credentials, rejected writes, partial failures.
8. **Isolate every test:** its own database, its own fake on its own port, no shared state.
9. **Keep integration tests fast and few** (the pyramid), separate them with a marker, and run them in CI.

15 Common Mistakes

Common Pitfall

Believing that passing unit tests mean the system works. The shop's unit tests pass with both planted defects (Section 3.1).

Common Pitfall

Testing everything end-to-end. Big-bang tests are slow, hard to diagnose (`assert 502 == 201`), and can even miss defects that a lower-level test catches (the buggy service passed every API test).

Common Pitfall

Doubles written from your own assumptions. The loose `responses` stub passed for the client that sends dollars.

Common Pitfall

Checking only the response. An order that returns `402` but leaves its stock reserved is still a defect. Look at the database, through a separate connection.

Common Pitfall

Calling real external services from tests. Slow, flaky, sometimes expensive — and you cannot make them fail on demand. Use a fake or a sandbox; keep real-service checks few and separate.

Common Pitfall

Shared test databases and fixed ports. The two most common sources of tests that pass alone and fail together.

16 Applying This to Your Semester Project

- Draw your system's components and **number its integration points**, as in Figure 2. Include databases, files, external APIs, and third-party libraries with I/O.
- Hold a short **design/API review** on those points with your team, using the checklist of Section 6. Record the findings in a table with an ID for each.
- For at least the riskiest findings, write an **integration test case specification** (Section 13) and automate it. Put the tests in `tests/integration/` with an `integration` marker.
- In your report, show the **traceability**: finding → scenario → test → result, and say which doubles you used where, and why.
- For Track A (existing open-source) projects: look for the project's own integration tests and fixtures first. Extending them is usually more valuable than starting new ones.

17 A Practical Mental Model

When you look at an architecture diagram, think:

> "Every arrow is a promise between two teams. Where is each promise written down, and which test checks it?"

When you review an interface, think:

> "Names, types, units, errors, credentials, transactions, timeouts. What would make the two sides disagree?"

When you write a test double, think:

"Am I writing down what the other side does — or what I hope it does?"

When an integration test passes, think:

"Did I check both sides of the interaction, and did I test what happens when the other side fails?"

18 Summary

Concept	In one line
Integration testing	Tests the interactions between components or systems; finds defects in the connections, not in the parts.
Big-bang	Connect everything at once; simple, but failures are hard to localize.
Top-down / bottom-up	Add components one layer at a time, using stubs below or drivers above.
Integration point	A place where two components must agree; the unit of integration test planning.
Design/API review	Examining interfaces, specifications, dependencies, and data flows to find integration risks early.
API testing	Requests, responses, status codes, formats, errors, and authentication, against the API contract.
Database testing	Connectivity, CRUD, constraints, transactions, and consistency, against a real database per test.
Service testing	External dependencies, including outages, timeouts, retries, and idempotency, against a fake or sandbox.
Test doubles	Replace what lies beyond the integration point; write them from the other side's specification.
Automation	Separate folder and marker, fixtures that assemble the system, isolated tests, CI.

Tool / command	Purpose
Flask `test_client()`	send requests to an API in-process, through its real routing[18]
`requests`	the HTTP client the shop uses to call PayFast[19]
`responses`	stub HTTP calls made through `requests`[20]
`werkzeug.serving.make_server`	run a fake service on a real local port
`sqlite3` + `tmp_path`	a real, isolated database file per test
`pytest -m integration`	run only the integration tests (marker in `pytest.ini`)

Key Idea

Unit tests check the parts; integration tests check the promises between them. Find those promises in the design, review them before you code, turn every risk you find into a test, and keep those tests running.

19 Practice Exercises

Exercise 1 — Integration Points

A library system has a web UI, a `LoanService`, a `CatalogRepository` (PostgreSQL), a `NotificationClient` that sends SMS through a third-party API, and a nightly job that imports new books from a publisher's CSV feed. (a) Draw the components and number the integration points. (b) For each point, say whether it is component or system integration, and what the contract is. (c) Propose an integration order, and say which stubs or drivers it needs.

Exercise 2 — Review an API Specification

A partner's API documents `GET /availability?date=2026-11-03` returning `{"slots": ["09:00", "09:30"]}`, with `429 Too Many Requests` when called more than 10 times a minute. (a) Using the checklist of Section 6, list at least five review findings (hint: time zones, empty results, pagination, error format, rate limits). (b) Turn three of them into test scenarios with an integration point, a trigger, and an oracle.

Exercise 3 — Hands-On: Refunds

In `demos/integration_testing/`, implement the refund feature of Section 13: extend the fake provider with `POST /v1/refunds`, then add the client method, the service method, and the API route. Write the five test cases IT-R1–IT-R5 first, watch them fail, then make them pass.

Exercise 4 — Fix the Known Issue

In the main suite, `test_failed_welcome_email_does_not_leave_a_half_registered_user` is a strict `xfail`. Change `UserRegistrationService.register()` so that it passes (two reasonable designs: delete the user if the email fails, or save the user with a "welcome pending" flag and retry later). Remove the marker, and explain your design choice. Which design is easier to test, and why?

Exercise 5 — Unexpected Dependency Behavior

PayFast returns `201` with the body `{"status": "pending"}` — no `id`. (a) What does `PaymentClient.charge()` do today? Write a `responses` test to find out. (b) What *should* it do? Change the client, and add a test against the fake provider as well.

Exercise 6 — Design Question

A teammate proposes deleting `test_api_topdown.py` because `test_api.py` "already tests the API, with real components". Using at least two ideas from this reading, argue for or against.

20 Discussion Questions

1. Why can a defect pass every unit test of every component, and still be a defect?
2. Compare big-bang and incremental integration on fault localization. Use the `assert 502 == 201` failure as an example.

3. What is the difference between a stub and a driver? Which one is a `pytest` test function, in bottom-up integration?
4. Why did the buggy order service pass every test in `test_api.py`? What does that say about relying only on end-to-end tests?
5. Why does the shop's database fixture use a file in `tmp_path` rather than `":memory:"`?
6. Why should a client not retry a `401`, but may retry a `503`? What makes retrying a payment safe?
7. A timeout is "not a failure but an unknown". Explain, and describe how a production system could resolve the unknown.
8. How can a mock or stub "agree with the bug"? How do fakes and contract tests reduce that risk?
9. Which of the reading's integration failures could have been found by a design review alone? Which could not, and why?
10. Where would you draw the line between integration tests and system tests (Week 10) for the shop?

21 Key Takeaways for SE 413/513

1. **Integration testing checks the interactions between components**, where unit tests — by design — cannot look.
2. **Integrate incrementally and by risk;** stubs replace what is below, drivers what is above.
3. **Integration points are the unit of planning.** List them, find their contracts, and test each one.
4. **Review designs and APIs early,** and turn every finding into a test, a change, or a documented decision.
5. **APIs, databases, and external services each have their own failure modes:** formats and status codes; constraints and transactions; outages, timeouts, and retries.
6. **Test doubles belong beyond the integration point,** written from the other side's specification.
7. **Automate:** a separate folder and marker, isolated fixtures, fast fakes, and CI.

22 References

- Integration testing as a test level; component and system integration; reviews as static testing.[2, 7]
- Integration strategies (big-bang, top-down, bottom-up, sandwich), stubs, and drivers.[5, 4, 3]
- Design and code inspections.[8]
- Integration defects at interfaces: the Mars Climate Orbiter.[9]

- Test doubles, fakes, and contract tests.[10, 11, 12, 13]
- The test pyramid.[14, 15]
- HTTP semantics and status codes.[16]
- Resilience to failing dependencies: timeouts and circuit breakers.[17]
- Tools: Flask, requests, responses, SQLite foreign keys.[18, 19, 20, 21]
- Larger tests in practice, and `pytest`.[6, 22]
- Course material on software testing principles and related testing practices.[1]

References

[1] Hasan, K. (2026). *Software Testing*. Grand Valley State University. https://github.com/mdkamrulhasan/software-testing

[2] International Software Testing Qualifications Board. (2024). *Certified Tester Foundation Level (CTFL) Syllabus*, v4.0.1.

[3] Ammann, P., & Offutt, J. (2016). *Introduction to Software Testing* (2nd ed.). Cambridge University Press.

[4] Jorgensen, P. C. (2013). *Software Testing: A Craftsman's Approach* (4th ed.). CRC Press.

[5] Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). John Wiley & Sons.

[6] Aniche, M. (2022). *Effective Software Testing: A Developer's Guide*. Manning Publications.

[7] IEEE. (2008). *IEEE Standard for Software Reviews and Audits* (IEEE Std 1028-2008).

[8] Fagan, M. E. (1976). Design and code inspections to reduce errors in program development. *IBM Systems Journal*, 15(3), 182–211. https://doi.org/10.1147/sj.153.0182

[9] Mars Climate Orbiter Mishap Investigation Board. (1999). *Phase I Report*. NASA, November 10, 1999.

[10] Meszaros, G. (2007). *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley.

[11] Fowler, M. (2011). *ContractTest*. https://martinfowler.com/bliki/ContractTest.html

[12] Fowler, M. (2018). *IntegrationTest*. https://martinfowler.com/bliki/IntegrationTest.html

[13] Pact Foundation. (2026). *Pact Documentation*. https://docs.pact.io/

[14] Cohn, M. (2009). *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley.

[15] Vocke, H. (2018). *The Practical Test Pyramid*. https://martinfowler.com/articles/practical-test-pyramid.html

[16] Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP Semantics* (RFC 9110). IETF. https://www.rfc-editor.org/rfc/rfc9110

[17] Nygard, M. T. (2018). *Release It! Design and Deploy Production-Ready Software* (2nd ed.). Pragmatic Bookshelf.

[18] Pallets. (2026). *Flask Documentation: Testing Flask Applications*. https://flask.palletsprojects.com/en/stable/testing/

[19] Python Software Foundation. (2026). *Requests: HTTP for Humans — Documentation*. https://requests.readthedocs.io/

[20] Sentry. (2026). *responses: A utility for mocking out the Python Requests library*. https://github.com/getsentry/responses

[21] SQLite. (2026). *SQLite Foreign Key Support*. https://www.sqlite.org/foreignkeys.html

[22] Okken, B. (2022). *Python Testing with pytest* (2nd ed.). Pragmatic Bookshelf.